Name: Nazrana Nahreen |
GitHub Repository: https://github.com/nazrana-nahreen/bangladeshi-taka-detection-api

Project Overview

This project implements an end-to-end deployment pipeline for a Bangladeshi banknote detection model. A YOLOv11 object detection model was trained to recognize eight Taka denominations (2, 5, 10, 20, 50, 100, 500, 1000), then wrapped in a REST API, containerized with Docker, and deployed to a public cloud platform.

Task 1: Model Integration & Inference Pipeline

Approach:
The YOLOv11n model was trained on the Bangladeshi Banknote Dataset (Kaggle) using a Tesla T4 GPU, over 50 epochs. Since the dataset consists of single, pre-cropped banknote images per class rather than multi-object scenes, full-image bounding boxes were generated programmatically for each image to enable YOLO-format training.

Results:

- Final validation mAP50-95: **0.995**
- Training time: ~2.6 hours (50 epochs, 8 classes, ~20,000 images)

Inference Pipeline:
A Python function (`predict_image`) loads the trained weights and performs single-image inference, returning class name, confidence score, and bounding box coordinates (x1, y1, x2, y2) for each detection.

[Insert Screenshot: GPU confirmation]
[Insert Screenshot: Training log — 50 epochs, final metrics]
[Insert Screenshot: Sample inference output — detected note with visualized bounding box]

Task 2: REST API Development

Framework: FastAPI + Uvicorn

Endpoints:

Method	Path	Purpose
GET	/	Health check

POST	`/predict`	Accepts an image, returns detections

Input: `multipart/form-data`, field name `file`, JPEG/PNG only.

Output (JSON):

```
json
{
  "filename": "test.jpg",
  "num_detections": 1,
  "detections": [
    {
      "denomination": "20",
      "confidence": 0.9949,
      "bbox": {"x1": 0, "y1": 0.1, "x2": 330.91, "y2": 152}
    }
  ]
}
```

Error Handling:

- `400` — missing or empty file
- `415` — unsupported file type (non-JPEG/PNG)
- `422` — missing/invalid request body (FastAPI automatic validation)
- `500` — internal inference error

Task 3: API Testing & Validation

The API was tested against multiple images covering different denominations, plus edge cases for invalid input handling.

Denomination	Predicted	Confidence	Result
20 Taka	20	0.9949	✅ Correct
2 Taka	2	0.9969	✅ Correct
2 Taka (out-of-dataset photo)	1000	0.5877	❌ Misclassified

(add remaining test results — 10/50/100/500/1000)

Edge case testing:

- Uploading a non-image file → returned `415 Unsupported file type`
- Sending a request with no file → returned `422 Validation Error`

[Insert Screenshots: Each test image + corresponding Swagger response]

Discussion on Accuracy:
The model achieves near-perfect validation metrics (mAP50-95: 0.995), which reflects the structure of the training dataset — every image contains a single, centered, pre-cropped note with a full-image bounding box, making the task closer to classification than true multi-object detection. When tested on an out-of-distribution image (sourced outside the training dataset, different lighting/angle), the model produced a low-confidence (0.59) misclassification for a 2 Taka note. This indicates that while the model generalizes very well within its training distribution, real-world deployment scenarios (varied lighting, angles, backgrounds, multiple notes per image) would likely require more diverse training data and true object-level bounding box annotations to sustain this accuracy level.

Task 4: Dockerization

Dockerfile approach:

- Base image: `python:3.10-slim`
- Dependencies installed in separate `RUN` layers (fastapi/uvicorn/pillow, opencv-python-headless, ultralytics) to leverage Docker's layer caching during iterative builds
- System-level dependency `libgl1`/`libglib2.0-0` installed via `apt-get` (required by OpenCV even in headless mode, for `ultralytics`'s internal image-processing calls)
- Application code and model weights copied in the final layers
- Port 8000 exposed; container runs via `uvicorn`

Build command:

bash
docker build -t taka-detector-api .

Run command:

bash
docker run -d -p 8000:8000 --name taka-detection-container taka-detector-api

Verification:

bash
docker ps

Confirmed container status `Up`, port mapping `0.0.0.0:8000->8000/tcp`. API tested successfully via `http://localhost:8000/docs` while running inside the container.

Task 5: Documentation & Project Structure

```
bangladeshi-taka-detection-api/
├── app/
│   ├── __init__.py
│   ├── main.py          # FastAPI routes, validation, error handling
│   └── inference.py     # Model loading and prediction logic
├── models/
│   └── best.pt          # Trained YOLOv11 weights
├── notebooks/
│   └── train_phase1.ipynb  # Phase 1 training notebook (Kaggle)
├── test_images/         # Sample images used for testing
├── Dockerfile
├── .dockerignore
├── requirements.txt
└── README.md
```

Full setup, build, and usage instructions are documented in `README.md` in the repository.

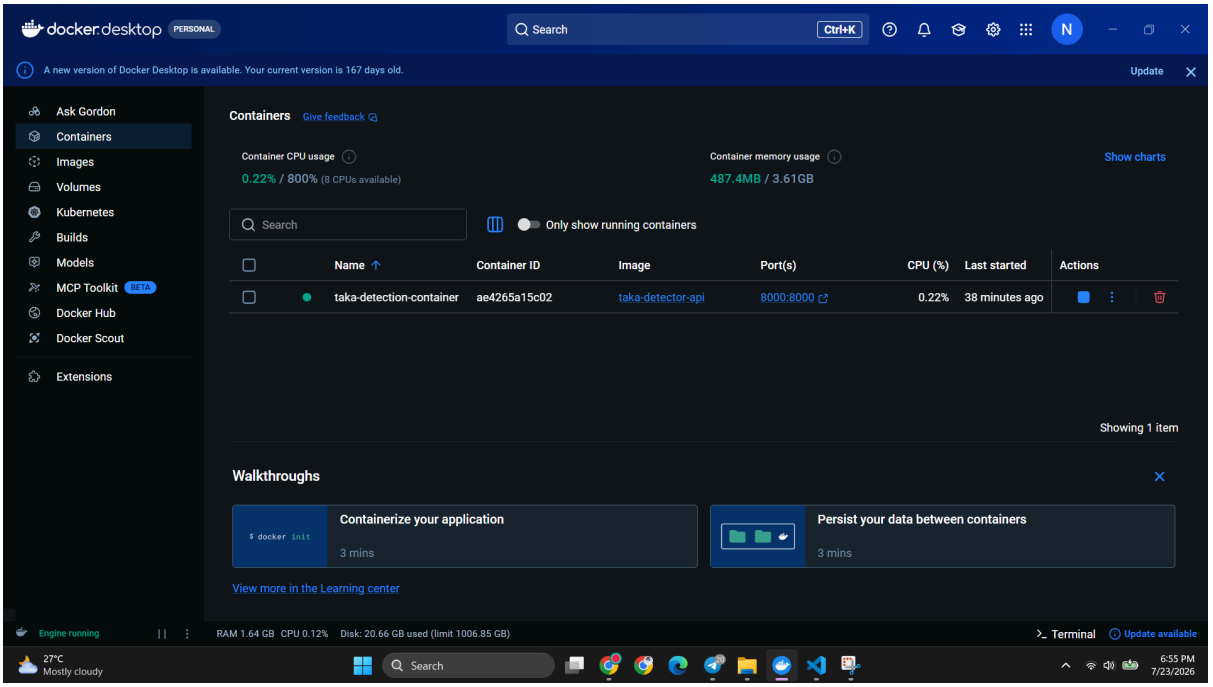
Bonus Task: Cloud Deployment

Platform: Render.com (free tier)

Note: Railway.app was initially attempted but the free trial had already expired. Hugging Face Spaces was also considered, but Docker SDK was found to require a paid plan on this account. Render.com's free tier was ultimately used — no card required, native Docker support.

Deployment steps:

1. Connected GitHub repository (`bangladeshi-taka-detection-api`) to Render via the GitHub integration.
2. Created a new Web Service, selected **Docker** as the runtime (auto-detected from the repository's `Dockerfile`).
3. Selected the **Free** instance type (512MB RAM, 0.1 CPU).
4. Deployed — Render built the Docker image directly from the repository and started the container.




```
Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 1/50      2.17G     0.1814       1.06     0.9025         27        640: 100% ──────────── 1063/1063 5.2it/s 3:260.2ss
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 94/94 5.3it/s 17.6s0.1s
             all       3000       3000      0.997      0.996      0.995      0.991

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 2/50      2.27G     0.1856     0.3677     0.8806         26        640: 100% ──────────── 1063/1063 5.9it/s 3:000.1ss
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 94/94 8.7it/s 10.8s0.1s
             all       3000       3000      0.954       0.95      0.992      0.944

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 3/50      2.27G     0.2064     0.3333      0.887         28        640: 100% ──────────── 1063/1063 6.1it/s 2:550.1ss
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 94/94 9.1it/s 10.4s0.1s
             all       3000       3000      0.992      0.988      0.994      0.993

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 4/50      2.28G     0.1855     0.2968     0.8812         29        640: 100% ──────────── 1063/1063 6.1it/s 2:540.2ss
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 94/94 8.9it/s 10.6s0.1s
             all       3000       3000      0.992      0.995      0.995      0.989

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 5/50      2.28G     0.1489     0.2389     0.8705         25        640: 100% ──────────── 1063/1063 6.1it/s 2:530.1ss
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 94/94 9.2it/s 10.3s0.1s
             all       3000       3000      0.999          1      0.995      0.995

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 6/50      2.28G     0.1282     0.2095     0.8645         25        640: 100% ──────────── 1063/1063 6.1it/s 2:550.1ss
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 94/94 9.2it/s 10.3s0.1s
             all       3000       3000      0.999          1      0.995      0.994

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 7/50      2.28G     0.1152     0.1902     0.8626         29        640: 100% ──────────── 1063/1063 6.1it/s 2:540.3ss
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 94/94 9.3it/s 10.1s0.1s
             all       3000       3000      0.999          1      0.995      0.995

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 8/50      2.28G     0.1071      0.175      0.861         26        640: 100% ──────────── 1063/1063 6.1it/s 2:540.2ss
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 94/94 9.0it/s 10.4s0.1s
             all       3000       3000      0.995      0.999      0.995      0.995

Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
 9/50      2.28G    0.09892     0.1627     0.8562         25        640: 100% ──────────── 1063/1063 6.1it/s 2:540.1ss
           Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ──────────── 94/94 9.3it/s 10.2s0.1s
             all       3000       3000          1          1      0.995      0.995
```

Bangladeshi Taka Note Detection Mod... Draft saved

File Edit View Run Settings Add-ons Help

Run All Code

```
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 94/94 9.0it/s 10.5s0.1s
                   all       3000       3000          1          1      0.995      0.995

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      47/50      2.28G     0.0178      0.024      0.825          8        640: 100% ━━━━━━━━━━━━ 1063/1063 6.2it/s 2:530.1ss
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 94/94 9.2it/s 10.2s0.1s
                   all       3000       3000          1          1      0.995      0.995

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      48/50      2.28G        inf    0.02256     0.8237          8        640: 100% ━━━━━━━━━━━━ 1063/1063 6.1it/s 2:530.2ss
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 94/94 9.2it/s 10.2s0.1s
                   all       3000       3000          1          1      0.995      0.995

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      49/50      2.28G        inf    0.02028      0.821          8        640: 100% ━━━━━━━━━━━━ 1063/1063 6.2it/s 2:520.1ss
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 94/94 9.1it/s 10.4s0.1s
                   all       3000       3000          1          1      0.995      0.995

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      50/50      2.28G    0.01367    0.01847     0.8228          8        640: 100% ━━━━━━━━━━━━ 1063/1063 6.2it/s 2:520.3ss
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 94/94 8.9it/s 10.5s0.1s
                   all       3000       3000          1          1      0.995      0.995

50 epochs completed in 2.575 hours.
Optimizer stripped from /kaggle/working/runs/taka_detector/weights/last.pt, 5.4MB
Optimizer stripped from /kaggle/working/runs/taka_detector/weights/best.pt, 5.4MB

Validating /kaggle/working/runs/taka_detector/weights/best.pt...
Ultralytics 8.4.104 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14912MiB)
YOLO11n summary (fused): 101 layers, 2,583,712 parameters, 0 gradients, 6.3 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 94/94 7.1it/s 13.3s0.1s
                   all       3000       3000          1          1      0.995      0.995
                    10        375        375          1          1      0.995      0.995
                   100        375        375          1          1      0.995      0.995
                  1000        375        375          1          1      0.995      0.995
                     2        375        375          1          1      0.995      0.995
                    20        375        375          1          1      0.995      0.995
                     5        375        375          1          1      0.995      0.995
                    50        375        375          1          1      0.995      0.995
                   500        375        375          1          1      0.995      0.995
Speed: 0.1ms preprocess, 1.0ms inference, 0.0ms loss, 0.7ms postprocess per image
Results saved to /kaggle/working/runs/taka_detector
```

Bangladeshi Taka Note Detection Mod... Draft saved

File Edit View Run Settings Add-ons Help

Run All Code

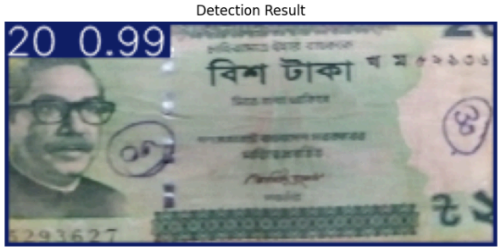
Draft Session off (run a cell to start)

```
plt.figure(figsize=(8,8))
plt.imshow(cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.title("Detection Result")
plt.show()
```

```
Testing on: /kaggle/working/data/yolo/images/val/20_20_(472).png

Detections:
{'class': '20', 'confidence': 0.9949, 'bbox': {'x1': 0.03, 'y1': 0.07, 'x2': 255.94, 'y2': 116.83}}
```

Detection Result



+ Code + Markdown

[2]:

```
import os
from IPython.display import FileLink

DATA_PATH = "/kaggle/input/datasets/rahnumatasnim1604103/bangladeshi-banknote-dataset/dataset"
files = os.listdir(DATA_PATH)

fifty_files = [f for f in files if f.startswith("50 (")]
print("Found 50 taka images:", len(fifty_files))
print("Sample:", fifty_files[:5])

if fifty_files:
```

Bangladeshi Taka Note Detection Mod... Draft saved

File Edit View Run Settings Add-ons Help

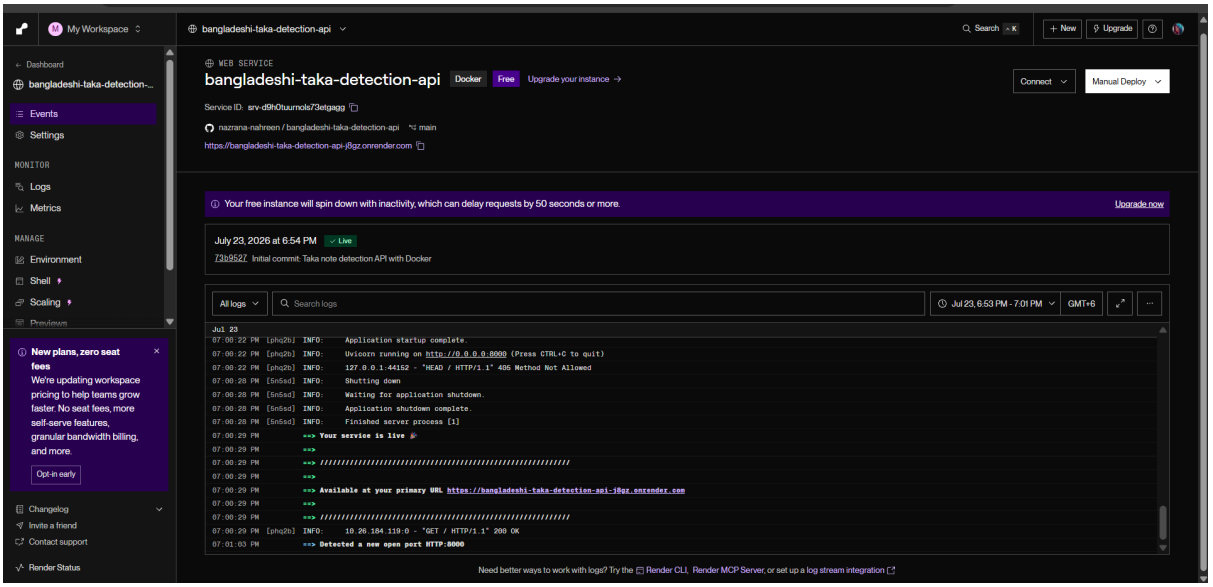
Run All Code Draft Session off (run a cell to start)

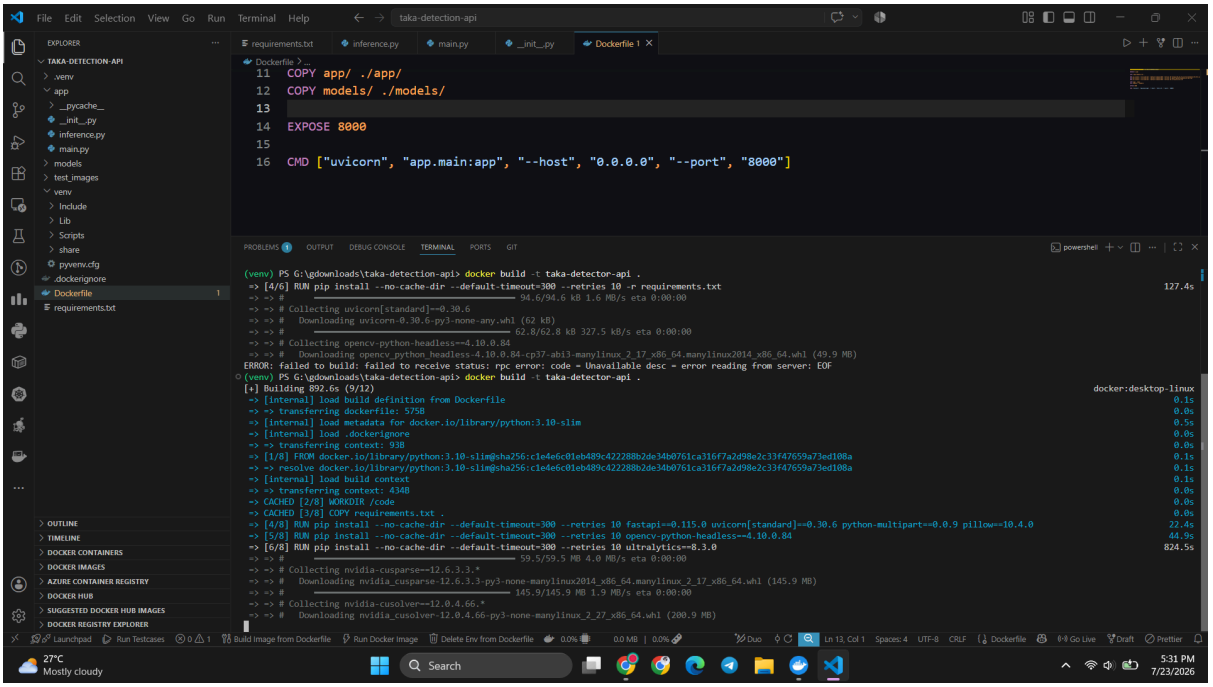
```
for i, denom in enumerate(denominations):
    matches = [f for f in files if f.startswith(denom + " (")]
    if matches:
        sample_file = matches[0]
        img_path = os.path.join(DATA_PATH, sample_file)
        img = Image.open(img_path)

        axes[i].imshow(img)
        axes[i].set_title(f"{denom} Taka")
        axes[i].axis('off')

        # copy to working
```

+ Code + Markdown





Bangladeshi Taka Note Detection API

1.0.0 OAS 3.1

/openapi.json

YOLOv11-based REST API for detecting Bangladeshi banknote denominations

default

GET / Root

POST /predict Predict

Schemas

Body_predict_predict_post › Expand all object

HTTPValidationError › Expand all object

ValidationError › Expand all object